Contents

A Dual-Domain Cognitive Architecture for Enterprise AI Governance: Decoupling Probabilistic Epistemic Inference from Deterministic Axiological Arbitration **1**
Abstract . . . 1
1. Central Thesis Statement . . . 2
2. Dissertation Structure and Chapter Outline . . . 3
Chapter 1: Introduction and the Organizational Design Impasse . . . 3
Chapter 2: Theoretical Foundations and Related Work . . . 3
Chapter 3: The MOEPA Dual-Domain Cognitive Framework . . . 4
Chapter 4: Unified Data Substrate and Specialized Compute Projections . . . 6
Chapter 5: Reference Implementation (the `moepa-lab` Stack) and System Orchestration . . . 7
Chapter 6: Empirical Evaluation and Validation . . . 7
Chapter 7: Systems Governance, Sovereign Infrastructure, and Deployment Patterns . . . 8
Chapter 8: Conclusion and Future Research Directions . . . 8
3. Mathematical and Logical Formulation for Chapter 3 . . . 9
Epistemic Proposal . . . 9
Admission Checks . . . 9
The Commitment Function . . . 10
Curation and Promotion . . . 10
4. Immediate Development Plan . . . 10
Phase 1: Formal Proposal and Chapter 1/2 Draft . . . 10
Phase 2: Empirical Validation and Test Data . . . 10
Phase 3: Substrate Architecture and Systems Writing . . . 11

A Dual-Domain Cognitive Architecture for Enterprise AI Governance: Decoupling Probabilistic Epistemic Inference from Deterministic Axiological Arbitration

A model proposes. It does not commit.

Working draft. Claim correction of the prospectus received 2026-10-03. Organizational-design hook added 2026-10-04: the ledger stores the business logic; the runtime implements it. Architecture references updated 2026-10-19: the specification cited is *The MOEPA Cognitive Architecture — Technical Specification* v3.1 (moepa-cognitive-architecture, `catalog/ARCHITECTURE.md`); the reference implementation is the moepa-lab stack, which replaces the earlier `philosophical-ai-lab`. The record serialization is JSON-LD against an owned context, normatively shaped by moepa-profile (formerly `spop-profile`); the earlier N-Quad framing is withdrawn.

Abstract

Enterprise AI fails first as organizational design. Eligibility rules, safety interlocks, and the thresholds that authorize or block a transaction are written into application code and into the sampling procedure. An engineer who blocks a transaction whenever a model's confidence falls below 0.85 has inscribed a strategic decision in an implementation artifact. The threshold is then invisible to the leadership that owns the constraint, unauditable as policy, and fused with the epistemic state that occasioned it. Prompt engineering, reinforcement learning from human

feedback, and raw token probability do not restore the separation.

This prospectus proposes MOEPA (Metrology, Ontology, Epistemology, Praxeology, Axiology) as the architecture that reverses that anti-pattern. The ledger stores the business logic. The technology implements it. A foundation model remains an uncommitted advisor: its output is an epistemic proposal, a tuple of candidate, graded confidence, metrological metadata (how each value was measured, and how far it can be trusted), evidence pointers, defeaters (facts or cues that reduce confidence or rebut the claim), and an explicit unknown. Commitment is a separate decision procedure. The candidate is admitted only when three deterministic checks pass against one snapshot: ontological grounding in discrete identifiers, praxeological preconditions and allowable transitions, and axiological non-compensatory constraints encoded as policy. Confidence is not an input to the axiological check, and it never overrides a deny. Inscribing a probability on an entity record is a category mistake. Uncertainty stays attached to the claim.

A decision is auditable when it records the policy snapshot identifier and the gate results. Data sovereignty, on this account, is retention of that cognitive history — lineage, refinement rules, and epistemic claims — in an open serialization: the MOEPA record, one triple with its epistemic, metrological, axiological, and praxeological envelopes as JSON-LD against an organization-owned context, stored in Apache Iceberg. A bare export of extensional rows does not preserve it. The prospectus formalizes the commitment function, specifies a reference implementation, and proposes an evaluation of the gating design. The gating design does not prove alignment, and it does not prove that the snapshot is correct or complete.

1. Central Thesis Statement

A model proposes. It does not commit. The prevailing failure of enterprise AI is an inversion of authority. Business strategy is implemented as control flow: a confidence cutoff, an eligibility branch, a block-or-allow literal in service code. The organization then cannot inspect, version, or amend the rule except by changing the program. Leadership has lost the steering wheel to the codebase.

MOEPA reverses the inversion. An epistemic proposal carries a graded confidence and does not commit a fact, an entity, or a permission. Three deterministic admission checks follow, in order: ontological grounding, praxeological preconditions, and axiological non-compensatory constraints. Commit only if all three pass. The rules those checks apply are stored in the ledger and exported as a policy bundle. The runtime evaluates the bundle. It does not contain the strategy.

A numeric threshold is the same inversion in miniature. A procedure that blocks a transaction when $\mathcal{P}(\hat{y} \mid x) < 0.85$ treats an epistemic state as an axiological permission and locates the permission in source code. If leadership adopts a threshold, the threshold is a versioned clause in $\mathcal{K}_{\text{policy}}$. The axiological check does not receive $\mathcal{P}(\hat{y} \mid x)$. Confidence never overrides a deny. This is a gating design. It does not prove alignment. Auditability means the decision records the policy snapshot identifier and the gate results. Cryptographic signing is future work.

The model may integrate evidence with graded confidence. Commitment to an entity, an authorized transition, or a non-compensatory constraint is a separate decision procedure. Tagging a passage with a MOEPA layer does not implement that procedure. A schema check, a graph lookup, and a policy evaluation can. MOEPA and Marr answer different questions. This prospectus does not cross them into one model.

2. Dissertation Structure and Chapter Outline

This section states what each chapter must establish, in the order the argument needs it. Chapter 1 states the problem. Chapter 2 reviews the literature the argument answers. Chapters 3 to 5 build the framework, the data substrate, and the reference implementation. Chapter 6 tests the design. Chapter 7 places it in governance and deployment. Chapter 8 concludes.

Chapter 1: Introduction and the Organizational Design Impasse

1.1 **The Organizational Design Impasse.** The enterprise alignment problem is the displacement of strategy into black-box code. Engineers write business rules as branches, thresholds, and guard clauses. A hosted model's token probability is then read as warrant for the same commitment. Prompt engineering, reinforcement learning from human feedback, and layer tags in the prompt do not install an admission check. Scope note, author observation, not a completed experiment: layer tags in the prompt were tried and did not enforce constraints, because the model treated the tags as additional tokens. A second author observation, not a completed study: production services commonly encode a confidence cutoff (illustratively, block below 0.85) as application logic. That cutoff is policy wearing the syntax of a program.

1.2 **The Dual-Nature Category Error.** Implementation noise does not force the goal or the rule layer to be a probability calculus. Stochastic signal processing is a fact about realization. Legal, operational, and ethical admission rules are a separate decision procedure, stored as policy and interpreted by the runtime. Governance is not a Marr level. Writing a machine-learning probability onto an entity record is a further category mistake, developed in §2.3: the probability is an epistemic state about a claim, and the ontological record is a discrete identifier plus the relations asserted of it.

1.3 **Architectural Scope and Contributions.**

- Formalization of the MOEPA (Metrology, Ontology, Epistemology, Praxeology, Axiology) cognitive stack as a gating design in which the ledger holds the rules and the technology implements them.
- The operational separation of an epistemic claim from an ontological entity record, so that uncertainty is never stored as a property of instantiation.
- Praxeological and axiological constraints as a versioned policy bundle, evaluated outside the model, so that leadership amends strategy by snapshot rather than by patching inference code.
- An Iceberg ledger of MOEPA records (JSON-LD against an owned context) and refinement rules, with a vector index, a graph database, and a policy engine fed by snapshot sync or export. Sovereignty is retention of that cognitive history, not possession of a row export.

Chapter 2: Theoretical Foundations and Related Work

2.1 **Marr's Tri-Level Hypothesis Revisited.** Marr's levels are computational, algorithmic/representational, and implementational (Marr, *Vision*, 1982). The personal level is not one of Marr's levels: his three levels do not include a level of personal reasons, so the chapter does not treat a person's reasons as a fourth Marr level. Physical noise at the implementational level does not imply that the computational goal must be a probability calculus. The same holds for the rule used at the algorithmic level. The chapter revisits McClamrock (1991) and the 2015 exchange in *Topics in Cognitive Science* (Peebles; Love; Hardcastle and Stewart). It sets both against the opposing view in Pillow (2024). The full chapter states what that opposing view claims and where the disagreement lies.

2.2 **Neuro-Symbolic AI Architectures.** The historical divide, as surveyed in this chapter, is between symbolic logic systems and connectionist/deep models. Symbolic systems have proved

brittle under real-world noise. Connectionist models, in the architectures surveyed, have not provided absolute constraint enforcement. The chapter states the evidence for each side, and its verdict motivates the dual-domain split in Chapter 3.

2.3 **Epistemic Logic vs. Ontological Realism.** A schema firewall: uncertainty attaches to a claim about an entity, not as an existence probability on the entity record. Inscribing $P(\text{entity } E \text{ exists}) = 0.4$ as the entity's existence is a category mistake. The identifier would then denote a degree of credence. The firewall assumes a closed world for the records it checks. A closed world, for this check, means that an assertion absent from the checked records is treated as false for that check, rather than as unknown. Open-world identity and temporal validity remain claims. The same firewall withholds confidence from the action rule: a threshold that authorizes or blocks is policy, stored with the bundle, or it is not a rule of the architecture. Chapter 3's ontological check (Ω) is built on this firewall.

2.4 **Non-Compensatory Constraints.** A non-compensatory constraint is a constraint that cannot be traded off against benefit: it holds even when violating it would help. Statutory prohibitions and safety interlocks are examples. This section does not prove that axiology is binary. Non-compensatory constraints are boolean. Other values are tradeoffs and stay outside the hard gate. The hard gate is the subset encoded as policy. It becomes the axiological check (Λ) in Chapter 3.

Chapter 3: The MOEPA Dual-Domain Cognitive Framework

3.1 **The Probabilistic Domain ("Continuous Evidence Integration").** This domain is Metrology and the graded part of Epistemology only.

- **Metrology:** Telemetry, confidence scoring, uncertainty calibration, calibration error bounds ($\mathbb{E}[|P - Y|]$), and empirical disagreement tracking.
- **Epistemology, graded part:** Evidence integration and a graded confidence on a candidate output. The model remains an uncommitted advisor.
- **Epistemology, beyond graded confidence:** Defaults, defeaters, source tracking, and an explicit unknown state, distinct from a uniform prior. These are part of the proposal. They are not a permission.

3.2 **The Deterministic Domain ("Discrete Admission").**

- **Ontology:** Ground-truth schemas, discrete identifiers, and RDF subject-predicate-object assertions checked against the snapshot's graph.
- **Praxeology:** Preconditions and allowable transitions. Inputs to those checks may be graded. The check itself returns pass or fail.
- **Axiology:** The hard gate. Non-compensatory constraints encoded as policy. Tradeoff values are outside this gate.

Ontology answers what is instantiated. Praxeology answers which transitions the current state permits. Axiology answers which outcomes are forbidden regardless of benefit. All three answers are data. Preconditions, allowable transitions, and non-compensatory constraints are rows and rules in the snapshot, exported into the bundle the engine evaluates. Graded measurements may be inputs to a praxeological check; the check still returns pass or fail, and the criterion of passage is the rule, not the model's confidence. The axiological check does not receive confidence at all. Amending a prohibition is a new snapshot. It is not a change to the sampler and not a patch to the service that merely invokes the engine. That is the return of the steering wheel: leadership edits $\mathcal{R}_{\text{pre}}$ and $\mathcal{K}_{\text{policy}}$; engineers maintain the interpreter.

3.3 **The Commitment Function.** Commit the candidate only when ontological grounding, praxeological preconditions, and the axiological hard gate all pass. Otherwise refuse, and record

which gate failed. The formula is stated in §3 below.

3.4 **Curation: How Deterministic Layers Earn Determinism.** The three deterministic layers are deterministic only after deliberate human curation. Curation, in this prospectus, means human review, editing, and approval before a definition is written into the layer the gates read. Ingestion alone does not produce a layer. A schema, a form, or a policy document can seed a candidate definition. It cannot complete one.

This ladder follows the bronze–silver–gold staging already used for Metrology, where material moves from raw ingest through human-enriched stages for different uses before it is trusted as stable ground. It is applied here to the definitions the gates read: Bronze is raw ingest: schemas, documents, and extracted rules, kept for lineage and as working material. Silver is human-enriched, coherent, and limited in scope. Silver is provisional: it may be reviewed and tested, but it is not yet the authoritative map the runtime reads. Gold is reviewed, tested, versioned, and authoritative. Only gold definitions are published into the snapshot and bundle the gates evaluate. "Gold" in this ladder is a curation stage. It is not the specification's Golden Master baseline, the locked policy baseline for the policy bundle.

Ontology shows why curation is needed. A Taxpayer illustrates the work. A Taxpayer is defined by identity conditions, the criteria for treating two records as the same entity across roles and over time, and by the properties and relations that constitute it. Simple attributes include a taxpayer identifier and filing status for a tax year. Nested entities carry their own ontologies: Income is composed of W-2 wages, interest, and dividends. Relations run to other tax-domain participants: spouse, dependents, and employers. Requiring a Social Security number (SSN) or taxpayer identification number (TIN) for a dependent does not claim the dependent is currently filing. It places the person inside the Taxpayer identity space, so the same entity can later occupy the filer role without loss of identity. An employer is likewise a role. It may be filled by an individual Taxpayer or by an Organization possessing a TIN, an employer identification number (EIN). In both cases the identifier is the hook that keeps the entity inside the shared tax-domain graph. Because the definitions state identity conditions, prospective roles, inverses, cardinality, and temporal scope, the result is a navigable graph rather than a pure tree. Traversal, completeness checks, and cross-entity reasoning follow the declared edges.

Schemas and existing data models can seed a candidate ontology by revealing what is already measured. They cannot complete it. Distinctions that matter, identity across roles, legal constraints, and composition versus association, are only partly present in any schema. Human work is required to close those gaps. Once it is done, membership, relations, and valid walks are categorical for given data.

Praxeology is not coextensive with a procedure manual. Statutory obligations, operational constraints, risk controls, and the structure of the ontology itself can require actions that a manual omits or sequences differently. Policy and procedure documents should guide the encoding of workflows into the layer. They should not remain external texts interpreted at runtime. Encoding produces explicit, executable structures, guarded transitions, process definitions, and action rules, that reference ontological types directly. At runtime the layer enforces the encoded workflows. The source documents remain audit artifacts. Raw extraction of steps from a manual yields only bronze material.

The same requirement applies to Axiology. Value statements, fairness constraints, and normative rules extracted from policy are bronze until they are refined into operational, inspectable criteria. Those criteria must be evaluable against ontological instances and praxeological actions.

Pipelines must embody the ladder. Bronze is automated extraction, producing candidate structures plus lineage only. Silver and gold are human-guided refinement. A model may propose

missing identity conditions, surface implicit constraints, draft workflow encodings, flag inconsistencies, or generate test cases. Every proposal is reviewed, edited, or rejected by a human before it is written into the layer. The model never commits directly to the deterministic store. Promotion versions a gold definition, cross-validates it against the other layers, and publishes it as the authoritative map for runtime traversal, workflow engines, and constraint checkers. Epistemology stands outside the ladder. It has no definitions to promote, and its claims remain graded proposals that the gates receive at request time and never read from the snapshot. A record's Epistemology envelope (generating agent, generation time, fingerprint, producer trust) is the warrant for that version: it travels unchanged through every stage and is never itself approved. Promotion approves the record's ontological, praxeological, and axiological content. It does not raise a confidence or turn a claim into a fact, and a model's proposal is never promoted by the model. Metrology supplies observed measurements. Ontology, Praxeology, and Axiology supply curated, deterministic answers to what a thing is, what must be done, and what ought to be valued.

Chapter 4: Unified Data Substrate and Specialized Compute Projections

4.1 **The Ledger.** Apache Iceberg, under an open table catalog, is the versioned, append-only ledger of record and the only system of record (specification §5.0.1). The unit of storage is the MOEPA record (specification §5.0): the ontological triple plus one envelope for each of the other four layers — Epistemology (generating agent, generation time, SHA-256 fingerprint), Metrology (measurement confidence, validity window), Axiology (classification marking), Praxeology (allowed and denied action classes) — on the same row, serialized as JSON-LD against an organization-owned context (moepa-profile). The triple is the assertion; the envelopes are the context of its instantiation, and they travel with it in the same row. Because the record is JSON-LD it is also an RDF dataset, so a graph projection can hold the triple under the record identifier as a named graph without a second representation; the ledger row, not the graph, is the record. A record is never updated in place; a change is a new row with the same identifier and a new fingerprint, and older rows remain readable. The ledger also stores the refinement rules that praxeology and axiology evaluate. Snapshot history supports replay of both the admitted world and the policy under which it was admitted. Object storage is the medium. No particular cloud vendor is a theoretical requirement. A Parquet export of rows without the envelopes and without the bundle is not the ledger.

4.2 **Projections and Snapshot Syncs.** The vector index, the graph database, and the policy engine's loaded bundle are projections or syncs from snapshots. They are not live mounts, unless a virtualization path is later implemented and named. Each projection must be emptiable and rebuildable from the ledger; the specification makes that rebuild an acceptance test, and the reference implementation runs it as one (`AG6` check).

- **Metrology:** Analytical SQL over Iceberg tables for drift and calibration monitoring.
- **Epistemic projection:** A vector index (Qdrant) loaded from a lakehouse table of embeddings keyed to the record identifier and fingerprint.
- **Ontological projection:** A graph database (Apache Jena Fuseki in the reference implementation) loaded from a snapshot, used for entity-relationship checks.
- **Praxeological and axiological projection:** Rules exported from a snapshot into an OPA bundle. OPA evaluates that exported bundle. It does not query Iceberg at decision time.

4.3 **Snapshot-linked Replay.** A decision stores the record identifiers it governed, the hash of the policy bundle in force, and the gate outcome; that is the `moepa.decisions` ledger (`DS5`) as implemented. The Iceberg snapshot id is specified for the same record and is not yet written by the reference implementation. Signing is future work.

Chapter 5: Reference Implementation (the `moepa-lab` Stack) and System Orchestration

The reference implementation is the moepa-lab stack, built to the enterprise capability catalog of the specification. It replaces the earlier `philosophical-ai-lab` scaffold. Each catalog capability (`DS1`–`DS8`, `AG1`–`AG8`) is one deployable unit with a declared contract and its own check, so the engine behind a projection, the gateway, or the model proxy can be replaced without touching the ledger or the gates (specification §5.2, "Capabilities deploy as units"). The checks prove the contracts hold; they are not the benchmarks of Chapter 6.

5.1 **Schema Firewall.** Epistemic fields (confidence, model id, p-value) do not appear as columns on the entity table. They stay on the proposal and on the decision record. In the record of §4.1 they stay in the Epistemology and Metrology envelopes and are never written into the ontological triple.

5.2 **The Sequential Enforcement Pipeline.**

- **Enforcement point:** One policy-enforcing gateway (`AG7`; Envoy with external authorization in the reference implementation) fronts every model and tool endpoint. It holds the request while an authorizer loads the governing record, asks the policy engine, and writes the decision to the ledger; only an allow is routed upstream. A denied request is an HTTP refusal the model process never sees. Any model or tool reachable by a path that skips the gateway is a side door, and the specification scores it as an automatic NO-GO unless covered by a dated, recorded exception.
- **Candidate generation:** A foundation model, hosted or self-hosted, produces the epistemic proposal. It is reached only through the gateway and a model translation proxy (`AG8`) that makes the model a replaceable host. A hosted service is an example epistemic engine, not part of the architecture.
- **Ontological verification:** Referenced entities and relations are checked in the graph synced from the snapshot.
- **Praxeological gating:** Preconditions and allowable transitions are evaluated with OPA Rego against the exported bundle. The check returns pass or fail.
- **Axiological gate:** Non-compensatory constraints in the exported bundle are evaluated with OPA Rego. The specification places the caller's clearance in the identity registry (`DS8`) as a stored fact, not a header the caller asserts. The reference implementation does not yet include that registry; clearance still arrives as a header. The record's classification marking is the Axiology envelope the rule reads. The check does not receive the proposal's confidence. Rego is the encoding of the business rule for that snapshot. The surrounding service is the interpreter. A confidence cutoff, if leadership has adopted one, appears as a clause in the bundle, versioned with the snapshot identifier, and is still not an input that can override a separate deny.

5.3 **Handling Rejection and Fallbacks.** Safe refusal states, feedback for re-prompting or resampling, and human-in-the-loop escalation. Refusal names the first failed gate and is written to the decision ledger with the policy hash, as an allow is.

Chapter 6: Empirical Evaluation and Validation

The benchmarks in this chapter are proposed. None has been run for this prospectus.

6.1 **Proposed Benchmark 1 — High-Confidence Boundary Violations.** Scenarios in which a model assigns high confidence (including 0.99 and above) to a hallucinated or non-compliant candidate. The expected observation is refusal at the first failed gate. This is a test to design, not a result.

6.2 **Proposed Benchmark 2 — Enterprise and Public Sector Workflows.**

- Statutory benefits eligibility determination.
- Income verification and programmatic audits.
- Separation of a claim about a person from that person's entity record.

6.3 **Proposed Measurement — Latency.** Compare evaluation of an exported OPA bundle with foundation-model sampling. Latency is a measurement to make. This draft reports no latency figure.

6.4 **Proposed Check — Snapshot-linked Replay.** Replay a recorded decision against the stored Iceberg snapshot id and bundle version under a simulated review. Signing is out of scope until it is specified.

Chapter 7: Systems Governance, Sovereign Infrastructure, and Deployment Patterns

7.1 **Sovereignty of Cognitive History, and Owned vs. Rented Gates.** Possession of raw tables is not data sovereignty. Sovereignty is the ability to recover, on infrastructure the organization controls, the epistemic claims, the lineage of each admitted assertion, and the praxeological and axiological rules that governed commitment. MOEPA records in an open Iceberg ledger provide that recovery: the triple and its four envelopes in one row, in an open serialization the organization's own context defines. A bare row export fails that test. It abandons the warrant. A monolithic platform whose ontology and action rules are recoverable only inside a vendor runtime also fails it. It keeps the steering wheel inside the platform. Delegating the policy gate to a commercial API provider is the same failure at deployment time. Hosting the epistemic engine is a deployment choice. The admission checks and the rules they read remain outside that engine. The reference implementation makes the owned-versus-rented line a packaging rule. The ledger, the policy baseline, the gate, and the gateway are owned units. The graph store, the vector index, and the model host are units that can be swapped. A swap is proven by the unit's own check and by the projection rebuild passing with no other unit restarted.

7.2 **Air-Gapped and High-Assurance Implementations.** Deployment patterns for isolated infrastructure, including self-hosted models, a local vector index, and a self-hosted policy engine evaluating an exported bundle. A particular cloud partition is an example, not a theoretical requirement.

7.3 **Regulatory and Standards Alignment.** Mapping the gating design to the NIST AI Risk Management Framework (AI RMF), ISO/IEC 42001, and federal AI safety directives. Mapping is not certification.

Chapter 8: Conclusion and Future Research Directions

8.1 **Summary of the Claim.** AI safety here is treated as an engineering gating problem in addition to training-time alignment. The gate is also where strategy is stored as policy data and executed by the runtime, after the epistemic claim has been refused entry into the ontological record. The gating design does not replace alignment work, and it does not prove alignment.

8.2 **Extensions to Multi-Agent Swarms.** Applying the same proposal-and-admission split to distributed agent networks. This is future work.

8.3 **Open Research Questions.** Automated synthesis of candidate encodings of the policy-encoded hard gate, for human review and promotion, checking generated Rego against the statute it came from, hardware-accelerated policy evaluation, and signing of snapshot-linked decision records.

3. Mathematical and Logical Formulation for Chapter 3

This section states the commitment function in symbols, so the formula can be checked against the outline in §2. It proceeds in four steps: the epistemic proposal, the three admission checks, the commitment function itself, and the curation condition on which the checks' determinism depends. A snapshot, in this section, means the versioned ledger state together with the policy bundle exported from it: the fixed input the gates read for one decision. This section states a gating design. It is not a proof of alignment, and it is not a proof that the snapshot is correct or complete.

Epistemic Proposal

Let $\mathcal{E}$ represent the epistemic state generated by the model over input $x \in \mathcal{X}$, where $\mathcal{X}$ is the space of inputs the model can receive:

```
\mathcal{E}(x) = \langle \hat{y}, \mathcal{P}(\hat{y} \mid x), \mathcal{M}, \tau \rangle
```

Where:

- $\hat{y}$ is the proposed candidate output or action.
- $\mathcal{P}(\hat{y} \mid x)$ is the graded confidence on that candidate.
- $\mathcal{M}$ is metrological metadata (calibration error, token log-probabilities, and related uncertainty on the observation).
- τ holds evidence pointers (the retrieval context). It also identifies the snapshot the proposal was conditioned on. In this notation, τ also carries the defeaters and the explicit unknown marker named in the Abstract (see §2 3.1).

An explicit unknown is part of the proposal. It records the absence of a warrant. It is distinct from a uniform prior, which manufactures a distribution in that same absence. It is not an extra factor in $\mathcal{P}(\hat{y} \mid x)$.

Admission Checks

The three checks run in the order Ω, then Π, then Λ. Each returns pass (1) or fail (0). Inputs to a check may be graded. The check result is boolean. Confidence never overrides a deny.

Ontological check (Ω):

```
\Omega(\hat{y}, \mathcal{G}) \in \{0, 1\}
```

$\mathcal{G}$ is the graph synced from the snapshot under review. $\Omega = 1$ when every entity and relation referenced in $\hat{y}$ is grounded in that graph. The check uses a closed-world reading of $\mathcal{G}$ (see §2 2.3): for this check, an assertion absent from the checked records is treated as false, rather than as unknown. It does not convert an open-world or temporal uncertainty into an existence probability on the entity record.

Praxeological check (Π):

```
\Pi(\hat{y}, \mathcal{S}_{\mathrm{current}}, \mathcal{R}_{\mathrm{pre}}) \in \{0, 1\}
```

$\mathcal{S}_{\text{current}}$ is the current state from which the transition starts. $\mathcal{R}_{\text{pre}}$ is the set of preconditions and allowable transitions in the snapshot. $\Pi = 1$ when the transition from $\mathcal{S}_{\text{current}}$ by $\hat{y}$ meets the preconditions and lies in $\mathcal{R}_{\text{pre}}$. Graded measurements may be among the inputs. The result is pass or fail.

Axiological check (Λ):

```
\Lambda(\hat{y}, \mathcal{K}_{\mathrm{policy}}) \in \{0, 1\}
```

$\mathcal{K}_{\text{policy}}$ is the policy-encoded subset of non-compensatory constraints (statutory prohibition, safety interlock) in the exported bundle. $\Lambda = 1$ when $\hat{y}$ satisfies that subset. Λ does not take $\mathcal{P}(\hat{y} \mid x)$ as an input. $\mathcal{K}_{\text{policy}}$ and $\mathcal{R}_{\text{pre}}$ are artifacts of the snapshot. A threshold or a prohibition that exists only as a literal in application code is outside the commitment function. No gate can record it, so no decision record can cite it. Tradeoff values are not part of $\mathcal{K}_{\text{policy}}$; they stay outside the hard gate (see §2 2.4).

The Commitment Function

```
\mathcal{A}_{\mathrm{commit}} =
\begin{cases}
\mathrm{Commit}(\hat{y}) & \text{if } \Omega = 1 \land \Pi = 1 \land \Lambda = 1 \\
\mathrm{Refuse}(\mathrm{reason}) & \text{otherwise}
\end{cases}
```

reason is the first failed gate, in order Ω, then Π, then Λ. In this formulation, a later gate is not consulted after an earlier failure: refusal names the first failure, so a later gate's result is not needed for the decision.

Ω, Π, and Λ are deterministic given the snapshot they read. They are not a proof that the snapshot is correct or complete.

Curation and Promotion

Ω, Π, and Λ are deterministic only when the snapshot they read is gold, in the sense defined in §2 3.4: reviewed, tested, versioned, and authoritative. Bronze material — raw ingest, schemas, documents, and extracted rules — is not in the snapshot the gates evaluate. It is kept for lineage only. A model may propose a definition, a workflow encoding, or a test case. A human reviews, edits, or rejects the proposal before it is written into a layer. Promotion is the human-approved step. It versions a definition, cross-validates it against the other layers, and publishes it into the snapshot. No proposal, however confident, is promoted by the model itself.

4. Immediate Development Plan

Phase 1: Formal Proposal and Chapter 1/2 Draft

- Formalize Chapter 1: Introduction and Problem Formulation
- Refine Chapter 2: Literature Review (Marr, including the revisits named in §2.1; neuro-symbolic systems; AI safety)
- Write Chapter 3 from the commitment function in §3, without treating the gates as a proof of snapshot correctness

Phase 2: Empirical Validation and Test Data

- Derive test cases from the `moepa-lab` stack: the per-capability checks under `deploy/capabilities/*/check.sh` (gateway refusal never reaching the model proxy, policy hash matching the loaded bundle, projection rebuild from the ledger) and the unit tests under `src/assembly/tests/`. These are contract checks; Chapter 6 benchmarks are built from them, not reported from them.
- Measure latency of foundation-model sampling (hosted or self-hosted) against evaluation of an exported OPA bundle, on the `moepa-lab` stack. Report the measurement; do not assume a figure in advance.
- Run the proposed failure-injection tests (high-confidence boundary violations) through the gateway, so a refusal is recorded in `moepa.decisions`.

Phase 3: Substrate Architecture and Systems Writing

- Document the MOEPA record schema (`moepa.quintuples`), the decision ledger (`moepa.decisions`), and the bundle-export pipeline as implemented
- Detail graph and Qdrant projection from the ledger, and the rebuild check
- Add the Iceberg snapshot id to the decision record so §4.3 replay is implemented, not only specified
- After the proposed benchmarks have been run, assemble Chapter 6 from those results